MathProofOps:
Receipted Mathematical Manufacturing for Process Intelligence
From Public Ontology to Lean-Checked Standing

Sean Chatman

Abstract

We present MathProofOps, a discipline for lawful mathematical artifact manufacture, and mfact, its Lean 4 manufacturing framework. The governing law is $A = \mu_B(O_B^*)$ and $R_B = \text{receipt}(A)$: inside a declared boundary B, raw observations and generated candidates have no standing. Only admitted observations O_B^* may participate in manufacture; only replayable receipts R_B prove consequence. This separates contact from consequence. Humans, templates, and LLM agents may produce candidates; only the Lean kernel, Lake build, axiom audit, sorry census, negative fixtures, and release manifest can confer standing.

As the first manufactured vertical we introduce procint, a Lean 4 library for process intelligence: Petri nets with finitely supported markings, workflow nets with soundness split into independent clauses, event logs and alignment-based conformance, and object-centric event logs following OCEL 2.0. The library is manufactured from a declared obligation graph: an RDF/Turtle catalog of structures, theorem surfaces, module dependencies, fixtures, citations, and registry surfaces is projected through SPARQL-in-Tera templates by ggen, then admitted or refused by Lean/Lake. Every failed admission is a typed refusal rather than a silent downgrade; every admitted declaration is recorded with its hash, axiom footprint, fixture status, and proven/stated standing. The contribution is not merely that procint exists; it is that procint is the first manufactured vertical of MathProofOps, and mfact is the Lean/Lake manufacturing apparatus that makes this vertical receiptable.

A second rail extends the manufacturing graph from domain semantics to implementation correspondence. Using Aeneas/Charon, bounded Rust cores from wasm4pm-style process execution can be extracted into proof-facing semantic images. These images do not prove Rust correct by extraction alone; rather, they create correspondence obligations between implementation objects and admitted procint semantics under declared abstraction functions. This gives process intelligence a proof class it otherwise lacks: kernel-checked implementation-correspondent process law, not merely "model proved, implementation tested."

The result is not a claim that arbitrary generated mathematics is automatically correct. Rice-style undecidability is respected rather than ignored: MathProofOps replaces the impossible question "is this arbitrary generated artifact semantically correct?" with the bounded, replayable question "did this candidate pass the declared admission gate inside boundary B?" More generally, public ontologies supply a domain's nouns; MathProofOps supplies standing. The method generalizes from process intelligence to any ontology-bearing field whose artifacts can be projected, admitted, receipted, and replayed. The evaluation section of this paper is generated from the release manifest rather than written by hand, and a kernel-checked *Standing Quadrature* witness closes the cross-product between the declaration catalog, the admitted corpus, the release manifest, the process evidence, and this paper's claims. We report exactly what was proven, what remains stated, what was refused, and what the release boundary excludes.

1 Introduction

Formalization today often resembles hand-fitted manufacturing: every theorem, lemma, import, and repair is shaped locally by expert craft. The result can be excellent, but it is not yet industrial. Mathematical manufacturing asks a different question: what must be standardized so that formal domains can be produced as interchangeable parts? In mfact, the interchangeable part is not merely a Lean declaration. It is a declaration plus provenance, admission status, axiom footprint, refusal mode, hash, manifest position, and replay path. A theorem without this surrounding receipt is hand-fitted metal; an admitted declaration with its release evidence is an interchangeable mathematical part. The manufactured object is not syntax — it is *standing*.

Recent work uses LLMs to draft formal content, which raises a question the kernel alone does not settle: what standing does generated material have? Kernel acceptance establishes that a term inhabits its stated type; it does not establish that the stated type is the intended one, that the statement is not vacuously weakened, or that the artifact as a whole is a faithful rendering of a target theory. Meek et al. [9] document these failure modes in detail: compilation masks incomplete multi-part statements, added weakening hypotheses, and parameter restrictions. Our position is compressed by one rule: *contact is not consequence*. LLM contact with a theorem is not consequence. A model may propose a proof term, a definition, or a module, but the proposal has no standing until admitted. mfact formalizes this separation: contact produces a candidate; admission produces consequence; the manifest records the receipt.

Our position is that the two concerns should be mechanically separated:

- **Generation is unprivileged.** Anything — a template engine, a human, an LLM — may produce a *candidate*. Candidates carry provenance and a content hash but no standing.
- **Standing is admission.** A candidate acquires standing only by passing a fail-closed gate: the Lean kernel accepts it, it contains no `sorry`, and its axiom footprint is pinned to an authorized set. Every failure is a typed refusal, enumerated in a closed inductive type.
- **Claims are scoped.** A release manifest records, per declaration, whether it is *proven* (kernel-admitted, axiom-audited) or merely *stated* (the formal statement exists; the proof obligation is open). The paper's own evaluation is rendered from the manifest, not asserted in prose.

We call this discipline *mathematical manufacturing*: the pipeline is the product as much as the library is. The statement–proof split, the typed refusals, and the manifest are what make an overnight, largely unattended run reportable at all.

Scope. This paper does not claim to formalize all of process intelligence completely, nor does it claim the manufacturing pipeline is applicable beyond the case study reported here. It introduces a mathematical manufacturing pipeline and demonstrates it by generating a first Lean 4 process-intelligence foundation from an existing typed implementation inventory. Table 1 makes the pipeline concrete: each stage names the artifact it produces, not a metaphor.

This paper makes four bounded contributions.

First, it defines *mathematical manufacturing* as a typed admission discipline for formal artifacts. Generated output, human patches, templates, and agent proposals are treated uniformly as candidates. Standing is acquired only through admission: kernel acceptance, no-sorry census, axiom audit, negative fixtures, manifest completeness, and replayable receipt (Section 2).

Manufacturing stage	Concrete artifact
Source inventory	typed Rust implementation (wasm4pm-compat)
Rendering	ggen SPARQL-in-Tera templates (render Lean modules from the TTL catalog)
Admission	Lean module creation (`lake env lean`)
Inspection	`lake build`
Refusal	typed `Refusal`, negative fixtures
Evidence	process trace, release manifest
Certification	`CertifiedRelease` object, axiom audit
Publication	generated paper evaluation section

Table 1: Mathematical manufacturing stages and their artifacts.

Second, it introduces mfact, a Lean 4 framework that makes this discipline explicit through candidate, refusal, admission, manifest, certified-release, and valid-objection types (Section 6). The framework is intentionally small: it does not attempt to replace the Lean kernel, but to make the pre-kernel and post-kernel standing surface explicit.

Third, it introduces procint, the first manufactured vertical of MathProofOps, produced by mfact: a process-intelligence library covering Petri nets, workflow nets, event logs, conformance, and OCEL-style object-centric logs (Section 7). The novelty is not only the domain coverage — to our knowledge, based on a bounded search of the Lean, Coq, and Isabelle/AFP library indexes at the time of the run, we found no prior public mechanization of workflow-net soundness or OCEL in these systems, a negative search result rather than a proof of absence — it is the coupling of domain formalization with an explicit manufacturing receipt. A prior hand-written mechanization, if found, would be adjacent but not equivalent unless it provides the same admission, refusal, manifest, axiom, fixture, and replay surfaces.

Fourth, it reports a manifest-generated evaluation (Section 13) closed by a kernel-checked Standing Quadrature witness. The paper's evaluation numbers are not prose claims; they are rendered from the release manifest, and a generated Lean module (`ProcInt.Release.Quadrature`) is refused by the kernel if the declaration catalog, the audited corpus, and the manifest ever diverge. This closes the loop from declaration to admission to receipt to replay, and makes the paper itself an artifact of the manufacturing pipeline it describes.

2 The Receipted Manufacturing Law

The governing law of the pipeline is

$$A = \mu_B(O_B^*), \qquad R_B = \text{receipt}(A)$$

where B is the declared boundary (the Lean packages, the declaration catalog, the manifest schema, the source inventory, the audited theorem set); O is raw observation — candidate output from a human, an LLM, a template, a repository, or an ontology fragment; O_B^* is admitted observation inside B: typed, checked, bounded, replayable; μ_B is the lawful manufacturing transform (ggen rendering, Lean admission, Lake build, audit), parameterized by the boundary it operates inside; A is an artifact with standing (an admitted theorem, module, manifest entry, or release); and R_B is the receipt (build log, manifest hash, axiom audit, negative fixture, replay evidence) that proves A is a genuine consequence of μ_B, not an assertion about it. Raw generated output is O, not O^*. Manufacture produces the artifact; the receipt proves consequence; replay proves the receipt is non-ornamental. *The pipeline exists to refuse the gap.*

The Four Evidence Kinds. We distinguish four classes of evidence within the manufacturing graph: (1) Formal evidence (Lean theorems and axiom audits); (2) Operational evidence (praxis-graphlaw benchmark and execution validation via rslab); (3) Correspondence evidence (Aeneas-extracted image to procint semantic equivalence proofs); and (4) Publication evidence (generated paper fragments and manifests).

In this architecture, ggen is not a code generator in the usual sense. It is the projection half of the manufacturing function μ: it maps a declared obligation graph into candidate Lean surfaces. Lean supplies the admission half. Projection without admission is not standing; admission without projection does not scale.

Rice boundary and manufactured decidability. Rice's theorem bars nontrivial semantic decision procedures over arbitrary programs. The manufacturing response is not to evade this boundary, but to change the object of judgment. MathProofOps does not decide arbitrary semantic truth of arbitrary generated code. It decides whether a bounded candidate has been admitted by a specific kernel, under a declared axiom policy, with a complete manifest receipt. The undecidable question "is this arbitrary generated artifact mathematically correct?" is replaced by the decidable question "did this candidate pass the declared admission gate inside boundary B?" Undecidability is not defeated; it is fenced, and bounded standing is manufactured inside the fence.

Status is algebra, not prose. The release distinguishes a closed vocabulary of standing values: Declared, Extracted, Stated, Proven, Refused, Blocked, BuildBroken, Unknown, Unsupported, and Alive. No transition from Stated to Proven is implicit or inferred from prose; Extracted is not proof; Unknown is not admitted; Unsupported is not the same as Refused. A generated artifact remains a candidate, whatever its origin, until an explicit admission step assigns it one of these values.

3 From Public Ontology to Domain Standing

The method is not specific to process intelligence. Process intelligence is the first vertical because its public objects — events, traces, objects, transitions, guards, effects, conformance obligations, and object-centric logs — already carry close mathematical structure. The underlying pattern is field-independent. Public ontologies supply a domain's nouns; they do not supply standing. ggen projects those nouns into candidate artifacts; Lean/Lake admits or refuses them; manifests record algebraic status; receipts and replay preserve consequence across time.

For a field F with public object language $\mathcal{O}_F$, raw observations O_F, admitted observations O_F^*, lawful manufacturing transform μ_F, artifact A_F, and receipt R_F, the MathProofOps law specializes exactly as in Section 2:

$$A_F = \mu_F(O_F^*), \qquad R_F = \mathrm{receipt}(A_F).$$

Public ontology supplies the field's nouns; MathProofOps supplies standing. Finance has FIBO/XBRL/LEI; healthcare has FHIR/SNOMED/LOINC; supply chain has GS1/EPCIS/PROV-O; manufacturing has SOSA/QUDT/ISA-95; software has SPDX/SBOM and extracted implementation images. In each case the platform claim is not that the ontology is new, but that ontology-grounded artifacts become admitted, receipted, replayable objects with standing. procint is this paper's demonstration vertical, not the boundary of the method.

4 Architecture: The Standing-Manufacturing Graph

The standing-manufacturing graph is organized around a five-rail architecture. Table 2 details each rail's source, admission boundary, and receipt form.

Rail	Source	Admission Boundary	Receipt Form
Formal	Lean catalog, corpus	Lean kernel, axiom policy	Audit log, print axioms
Process-law	mfact core definitions	Conformance obligs.	covers proof, ValidObjection
Benchmark	praxis execution	rslab schemas	JSON/TOML result receipt
Correspondence	Rust crate source	Aeneas extraction	build_verif.py check
Paper	Manifest status	ggen sync	Manifest foldHash, regen-check

Table 2: The Five-Rail standing-manufacturing architecture.

5 Related Work

Formal libraries and their pipelines. Mathlib [16] is the reference point for large-scale collaborative formalization in Lean; its quality regime is human review plus continuous integration. CSLib [2] extends the Lean ecosystem toward computer science, including labeled transition systems, traces, and bisimulation, which procint builds on for its semantics layer. Lean 4 itself is described by de Moura and Ullrich [6]. The closest neighbor to mfact is the agent pipeline of Meek et al. [9], which formalizes numerical analysis with coding agents and audits quality beyond kernel acceptance. We differ in mechanism rather than diagnosis: where their audit is a post-hoc quality assessment over agent output, mfact is a *typed pipeline* — candidates, refusals, and admissions are Lean data types, the statement corpus lives in an RDF catalog curated by LLM-assisted lanes (each candidate kernel-verified before being recorded, rather than accepted on the strength of agent output alone), and the proven/stated split is enforced by the release gate rather than estimated afterward. The two approaches are complementary: their semantic audits address exactly the residual risk our pipeline does not eliminate, namely that a curated statement, though kernel-admitted, is itself a faithless or vacuous rendering of the intended theorem.

Lean 4 automation infrastructure. mfact's trust boundary sits above an already-scrutinized kernel: Lean4Lean provides an independent, kernel-in-Lean typechecker whose verification effort has already located and fixed a soundness bug in the reference implementation [5]. The two dominant patterns for wiring language-model or solver automation into Lean 4 are in-process integration, as in Lean Copilot's LLM inference over an FFI boundary [15], and external bridging to automated theorem provers, as in Lean-auto [14]. mfact adopts neither: candidates from any producer, language model or otherwise, are untrusted data admitted only through the gate of Section 6, so no automation tool sits on the trusted path regardless of how it is wired in. Infrastructure for premise and declaration search [1] and for large synthetic theorem corpora [22] addresses problems procint does not currently have, since its statement corpus is curated from the RDF catalog rather than mined or synthesized.

Process intelligence. The mathematical canon procint formalizes is due to a small number of primary sources. Petri-net structure theory — state equation, invariants, boundedness, liveness — follows Murata's survey [11]. Workflow nets and their soundness are van der Aalst's [17]; the

decidability and classification landscape is surveyed in [20], and the process mining setting in [18]. Alignment-based conformance checking follows Carmona et al. [4]. Object-centric event data follows the OCEL 2.0 specification [3] and object-centric Petri nets [19]; partially ordered workflow models follow POWL [8]. None of these, to the best of our search, had a prior mechanization in a proof assistant.

Code-to-proof extraction. The correspondence rail of Section 12 extracts an external Rust crate into Lean via Aeneas [7], whose Charon front end [13] separates syntax recovery from semantic translation. Aeneas targets whole Rust programs with a full memory model; our pilot uses it far more narrowly, on a single dependency-free arithmetic module, and treats its output the same way as any other candidate in this paper's pipeline — extraction produces a Lean statement, never a proof, and standing still requires kernel admission of a hand-authored correspondence theorem against that extraction. This rail addresses a proof class ordinary process intelligence lacks: without it, a Lean process model may be proven and a Rust executor may be tested, but their relationship remains prose, naming, convention, and examples. Extraction changes the object class, so that implementation and semantics can be related by a kernel-checked theorem rather than by documentation.

6 The mfact Framework

mfact is deliberately small: seven modules and a CLI. Its job is to make the pipeline's epistemics first-class Lean objects. mfact is not the whole system: it is the Lean/Lake standing apparatus inside a larger standing-manufacturing graph.

Candidates and refusals. A `Candidate` is content plus provenance (producer identity, content hash); producers are untrusted by construction. `Refusal` is a closed inductive type — `kernelRejected`, `sorryPresent`, `unauthorizedAxiom`, `missingEvidence`, `malformedModel`, `invalidFixture` — so a failed admission is always a value that can be logged, hashed, and re-queued, never an untyped error string. There is no default case: an unrecognized failure mode is itself a refusal to classify, not a silent pass.

Admission with a covers proof. An `Admission` bundles the evidence for one candidate: the kernel acceptance record, the sorry census, the axiom audit output, and a `covers` field — a proof term witnessing that the evidence entries cover the obligations the gate imposes, so an admission cannot be assembled with a missing check. The gate function `admit` is total: it returns either an `Admission` or a `Refusal`, with no third outcome.

Manifest, release, objections. A `Manifest` (JSON-serializable via `ToJson`) lists artifacts with hashes, axiom sets, fixture results, and the proven/stated status of each declaration. A `CertifiedRelease` pairs a manifest with its audit obligations. `ValidObjection R` is a closed inductive family whose constructors are the only admissible objection forms against a release `R` (an unaudited theorem, a missing hash, an unauthorized axiom, a failing fixture); each constructor demands evidence. The per-release theorem

```
theorem no_valid_objection :  IsEmpty (ValidObjection R)
```

is discharged by refuting each constructor against the corresponding checked field of `R`. For the release described in this paper the theorem is proven and, per `#print axioms`, depends on no axioms at all — not even the three standard Lean/Mathlib axioms. The theorem's scope is exactly its constructors: it says the *enumerated* objection forms are refuted, not that the release is beyond all criticism.

7 procint: Process Intelligence as Process Law

In this paper, process intelligence does not mean dashboarding over event logs. It means a mathematical substrate for deciding, replaying, and explaining process standing: what happened, what was admitted, what was refused, which object relations were involved, which conformance obligations passed, and which receipts authorize the resulting claim. procint depends on Mathlib and CSLib and is organized as follows (module map abridged):

Namespace	Content
`ProcInt.Foundations`	metrics as $\{q : \mathbb{Q} \mathbin{//} 0 \le q \wedge q \le 1\}$, identifiers, LTS
`ProcInt.Petri`	nets, firing, reachability, state equation, invariants, boundedness, liveness, OCPN, stochastic nets, unfoldings
`ProcInt.Workflow`	WF-nets, short-circuit construction, soundness
`ProcInt.Logs`	events, traces (monotone timestamps), event logs, XES
`ProcInt.Models`	process trees, POWL, DFGs, causal nets, BPMN, Declare
`ProcInt.Conformance`	moves, alignments, cost, fitness, token replay
`ProcInt.Ocel`	OCEL 2.0, E2O/O2O relations, flattening, lifecycles
`ProcInt.Ocpq`	object-centric process queries
`ProcInt.Analytics`	temporal, causality, correlation, cubes, streaming
`ProcInt.Registry`	algorithm and breed registries (cross-product tables)

Petri nets. A net is places, transitions, and weighted flow over finite types; a marking is a finitely supported function $M : P \to_f \mathbb{N}$, which unifies the weighted and (via richer codomains) colored cases and gives the marking algebra of Murata [11] — firing, the state equation $M' = M + A^{\mathsf{T}}u$, and place/transition invariants — as Finsupp lemmas with truncated subtraction.

Workflow nets and soundness. A `WfNet` carries a Petri net as a field (composition, not extension) together with the source-place, sink-place, and connectivity conditions of van der Aalst's Definition 6–7 [17]. Following the observation that the classical soundness clauses are logically independent, soundness is *not* a single conjunction-shaped definition but three separately named predicates — option to complete, proper completion, and no dead transitions — with `Sound` as their explicit conjunction. This keeps partial results honest: a lemma about one clause is stated about that clause.

The crown-jewel target of the run is van der Aalst's soundness characterization (Lemma 8 [17]): soundness of a workflow net iff liveness and boundedness of its short-circuited net. The finite transition hypothesis is necessary: the repository includes a Lean-admitted infinite-transition countermodel, `WfNet.infinite_transition_countermodel_sound_not_bounded`, showing that the unbounded statement fails under infinite transitions. The repaired crown theorem therefore states the equivalence under $[FiniteT]$.

Conformance. Alignments follow Carmona et al. [4]: moves are synchronous, log-only, model-only, or silent; costs induce optimal alignments; fitness is a Between01 value by construction, so boundedness of the metric is a type-level fact rather than a theorem obligation.

OCEL 2.0. Object-centric event logs follow the OCEL 2.0 specification [3] (Definitions 1–2): typed events and objects, event-to-object and object-to-object relations with qualifiers, and attribute maps; flattening onto a classical log and object lifecycles are derived operations. Object-centric Petri nets follow [19].

Canon status. Table 3 states, per area, whether procint v1 formalizes it, formalizes a seed instance of it, or leaves it to future work. The claim of this paper is a first combinatorial process-intelligence foundation, not a complete formalization of process intelligence; per-declaration status (proven/stated) within each formalized area is in the generated evaluation (Section 13).

Area	v1 status
Transition systems, traces, reachability	formalized (Foundations, Petri)
Petri nets (marking, firing, invariants)	formalized
Workflow nets and soundness	formalized; short-circuit theorem partial (Sec. 13)
Alignment-based conformance	formalized (moves, cost, fitness)
OCEL 2.0 event/object relations	formalized
Object-centric Petri nets	seed instance
Process trees, POWL, DFGs, BPMN, Declare	seed instances
Object-centric process queries (OCPQ)	seed instance
Temporal/causal/streaming analytics	seed instances

Table 3: Process-intelligence canon coverage in procint v1.

8 The Manufacturing Run

The run that produced procint has five moving parts.

The TTL catalog as canonical source. The canonical source of procint is not a tree of `.lean` files but an RDF/Turtle declaration catalog (`procint:` vocabulary): one record per declaration, carrying its Lean body (`leanCode`), its module, its proof status, its axiom-audit expectation, and citations back to the primary literature. Declaration bodies enter the catalog by curation — LLM-assisted lanes draft them and each candidate is kernel-tested before being recorded — so the catalog holds verified candidates, not free-form generation. The cross-product surfaces (registries of algorithms and model breeds) are fully data-driven rows of the same graph.

SPARQL-in-Tera rendering. The ggen engine renders Lean modules from templates whose frontmatter carries SPARQL `SELECT` queries against the catalog and whose bodies are Tera templates over the result bindings. One template yields one `.lean` file per `procint:Module`; separate templates render the root import index, the registry tables, and the axiom-audit file — for every audited theorem, a `#guard_msgs in #print axioms` block pinned to `[propext, Classical.choice, Quot.sound]`, so any transitive `sorryAx` or unauthorized axiom breaks the build. The rendered corpus is a *candidate artifact*: it is not trusted because it was rendered, and

ggen certifies nothing. It acquires standing only through kernel admission. In one line: ggen renders, Lean admits, mfact certifies.

LLM lanes as untrusted producers. Module families (Petri core, workflow, logs and conformance, OCEL, models, analytics, fixtures) run as independent lanes with no barriers between them. Each lane's agent hand-drafts the actual structures, definitions, and proofs for its module family, but writes nothing directly into the library: every piece of Lean code it proposes is first kernel-tested via `lake env lean` on a scratch file, and only a declaration that passes this check is recorded — as a `leanCode` string literal, together with its proof status — in the lane's catalog fragment. The catalog records what the lane authored and what the kernel confirmed; it authors nothing itself. Statements the lane cannot prove ship as `sorry`-bearing stubs marked *stated* in the fragment.

Admission and the repair loop. An integrator loop repeatedly merges fragments, re-renders, and runs `lake build`. Build errors are parsed and returned to the owning lane as repair context, with bounded retries per stub and per lane; exhausted stubs are downgraded — the statement stays in the library, the claim drops to *stated*, and the declaration leaves the audited set. Nothing is deleted and nothing passes silently: the terminal state of every stub is either kernel-admitted or an explicit downgrade receipt.

Certification. The release gate rebuilds both packages, builds the axiom-audit files, runs the negative fixtures (`#guard_msgs (error)` blocks, the Lean analog of compile-fail tests), and emits the release manifest with content hashes. A negative control — injecting a `sorry` into a scratch copy of an audited theorem — must cause the gate to refuse; a gate that cannot fail certifies nothing.

9 praxis-graphlaw: Executable Law-State Evaluation

The praxis-graphlaw engine (version v26.7.5) is an executable law-state engine that evaluates the operational conformance of transition systems and process models. Rather than operating as a custom verification engine built from scratch, praxis-graphlaw is a derivative work extending the RDF/N3 reasoner roxi (originally developed by Bonte et al.). It is integrated into the generation and validation lifecycle of ggen through the `GraphEngine` abstraction boundary, specifically concrete implementations of the `GraphLawStore`.

The engine models law-states directly as Resource Description Framework (RDF) triple graphs, using Turtle (.ttl) format serialization. Operational logic and state transitions are defined using Notation3 (N3) rules and stratified Datalog, allowing for deterministic materialization, solution generation, and query evaluation via the reasoner's `materialize`, `prove`, and `solve` interfaces.

State conformance and safety properties are enforced via W3C SHACL (Shapes Constraint Language) and ShEx (Shape Expressions) constraint schemas, which operate as executable transaction-path admission gates. The engine exposes these gates via the `validate_shacl` and `validate_shex` functions. Safety violations and negative conditions are captured as N3 logical denials of the form `{ body } => false.`, where any deduction of falsity immediately triggers a transaction refusal.

At the integration layer with ggen, graphlaw's validation results are translated into a structured, typed refusal vocabulary ranging from `FM-LAW-001` through `FM-LAW-013`. For example, an N3

logical denial violation produces an `FM-LAW-011` refusal, while a SHACL shapes non-conformance results in an `FM-LAW-013` refusal. This structured failure reporting constitutes an independent, executable instance of the MathProofOps law-state pattern.

Fast evaluation is a critical design requirement for these gates, as they sit directly on the transaction-path execution loop of the process engine. High latency during validation would directly degrade transaction throughput. While the primary objective is low latency, we measure and record these performance characteristics empirically in Section 13.3 to verify that the overhead of validation remains within acceptable limits.

10 rslab: Empirical Evidence Rail

The rslab framework acts as the empirical evidence rail for MathProofOps. It sits in parallel with the formal proof and correspondence rails, ensuring that measured performance and test execution data are receipted and ledgered deterministically. The governance of this rail is defined by the following doctrine:

> *praxis executes.*
> *rslab measures and receipts.*
> *mfact admits formal standing.*
> *paper renders admitted/receipted claims.*
>
> *rslab is not a proof engine.*
> *rslab is an empirical evidence rail.*

Within this rail, raw telemetry and outputs from execution are treated as unadmitted observations (O). Once these observations are processed by validation scripts, checked against JSON schemas, and bound to a cryptographic content receipt, they become schema-validated admitted observations (O^*). Downstream LaTeX paper fragments are then generated directly from these receipts to produce the final paper artifacts with standing (A).

Every experiment is declared in the central experiment manifest (`rslab/manifest.toml`) using the status ladder pattern. Telemetry outputs are validated against JSON schemas, such as `benchmark_result.schema.json`. The resulting TOML receipts mirror the metadata structure of `verif-receipt.json`, containing the execution builder, toolchain specifications (e.g. pinned compiler versions), git commits, and BLAKE3 hashes of raw captures. Crucially, in accordance with the determinism requirements, these receipts contain no wall-clock timestamps in their body.

The fragment generation pipeline is strictly fail-closed: if the processed results file or the validating TOML receipt is missing or has a hash mismatch, the builder script exits non-zero with an `RSLAB_EVIDENCE_MISSING` refusal, halting the build rather than allowing blank or stale placeholders to be compiled.

Currently, rslab defines a readiness contract capturing throughput and latency metrics for core graphlaw operations. However, profiling data is not available. This limitation is a direct reflection of the development environment: no profiling or flamegraph tooling exists in the praxis workspace. This absence is explicitly declared in the receipt caveats rather than silently ignored.

We explicitly distinguish rslab's role from formal verification. Where the correspondence rail (Section 12) proves a bounded relationship between an implementation and admitted semantics, the rslab rail records what a system measurably does, without claiming that measurement constitutes proof of anything beyond itself.

11 Use of Generative AI Tools

Claude Code and related language-model tooling were used to generate candidate patches, proof sketches, ontology fragments, documentation drafts, and release metadata during the run described in Section 8. These outputs were treated throughout as untrusted candidate artifacts, per the separation in Section 6: they were admitted into procint only when accepted by the Lean kernel, the sorry census, the axiom audit, schema validation against the `procint:` ontology, and the negative-fixture and release-gate checks of Section 8. No language-model output is part of the trusted base (Table 1); the trusted base is the Lean kernel, Mathlib and CSLib at their pinned revisions, and the `lake`/`ggen` toolchain. The author reviewed and accepts responsibility for all submitted content, code, citations, and claims in this paper and in the accompanying repository.

This separation is not precautionary in the abstract: general-purpose language models have a documented Lean 3/Lean 4 syntax-confusion failure mode, generating deprecated Lean 3 constructs despite explicit Lean 4 instructions, and GPT-4-class and substantially smaller models have been reported at comparable, low accuracy on the MiniF2F-Valid benchmark [10]. Approaches that fine-tune a general-purpose model toward Lean 4 specifically [21] or pair a language model with solver-guided refinement [12] trade some of this untrusted-candidate distance for higher raw acceptance rates; mfact does not take that trade in the run reported here, since every candidate — however produced — passes through the same kernel-and-audit gate regardless of its source's measured reliability.

12 Implementation Correspondence with Aeneas

Beyond procint's own catalog, mfact opens a second manufacturing rail: implementation correspondence. Where Section 7 manufactures domain semantics from a declared obligation graph, this rail manufactures obligations that relate a Lean declaration to code extracted from an *external*, unverified Rust crate (`wasm4pm-core`) via Aeneas [7] and its Charon front end [13]. The first instance of this rail is deliberately scoped to one obligation — token-replay counts correspondence, "D1" — not because the rail is narrow in principle, but because a single bounded instance is what standing requires before the rail generalizes. It follows the same status ladder as the rest of this paper's standing (declared, extracted, stated, proven), computed by a dedicated builder (`scripts/build_verif.py`) from observed evidence: a charon/aeneas pipeline receipt, a `lake build` of the extracted module against procint, and an axiom-print check on the resulting theorem. The crate's `ReplayCounts{produced, consumed, missing, remaining}` counts and its integer `fitness_num`/`fitness_den` functions are extracted verbatim; an abstraction function lifts the extracted (proof-free) struct into procint's `ReplayCounts` (which carries `missing` $\leq$ `consumed` and `remaining` $\leq$ `produced` as proof fields) given those two inequalities as hypotheses. The correspondence theorem does not assert that the crate's integer ratio and procint's exact-rational fitness are equal — they are structurally different formulas (a single integer fraction versus an average of two ratios) — only that each computes correctly from the same underlying counts; forcing a false numeric equality between them was rejected in favor of this honest, weaker, but PROVEN statement. Table 4 is rendered by that same builder from its own receipt; it is not written by hand.

Obligation	Status	Evidence
token_replay_counts_corr	PROVEN	full ladder evidence present (extracted, stated, proven)

Table 4: Correspondence-factory obligation status, computed from observed charon/aeneas/lake evidence by `scripts/build_verif.py` (status ladder: DECLARED < EXTRACTED < STATED < PROVEN; never hand-set past DECLARED). See release/verif-receipt.json for the full evidence trail.

13 Evaluation

13.1 Formal Standing Evaluation

Every number in this section is rendered from the release manifest (`release-manifest.json`) of the run; none is written by hand. The generation pipeline fails closed: if the manifest is absent or stale, this section does not render and the paper does not build for release.

Metric	Value
Lines of Lean (`ProcInt/`)	11,269
Declarations recorded in the ontology	397
Modules	53
Theorems kernel-admitted and axiom-audited (*proven*)	197
Statements formalized but not discharged (*stated*)	7
Definitions, structures, and test oracles	193

Table 5: Corpus size and the proven/stated/definition split, as recorded in the release manifest.

Axiom footprint of proven theorems	Count
No axioms	37
`propext`	28
`Quot.sound`, `propext`	26
`Classical.choice`, `Quot.sound`, `propext`	106
Total proven	197
Unauthorized axioms found	0
Transitive `sorryAx` found	0

Table 6: Axiom-audit result over all 197 proven theorems. Every set is a subset of the trusted {`propext`, `Classical.choice`, `Quot.sound`}; `no_valid_objection` (Section 6) is the strongest case, depending on none of them.

The two stated (unproven) declarations. Both are formal statements, not theorems: `WfNet.sound_iff_shortCircuit_live_bounded_statement` (the crown-jewel target, Section 7) and `BranchingProcess.isUnfoldingOf_statement` (an unfolding-correctness statement in the Petri-net seed lane). Per `crownJewel_status`, the crown-jewel target's release-time status is `"stated"`: neither direction of the iff is discharged in this release. Two supporting lemmas about `WfNet.Sound` (`reaches_final`, `enabled_of_transition`) *are* proven and audited (Table 6).

Fixtures. `ProcInt.Fixtures.Positive` and `ProcInt.Fixtures.Negative` both build; the negative fixtures are `#guard_msgs (error)` blocks whose docstring is the exact error Lean produces for a malformed `ReplayCounts` (missing token count exceeding consumed) and a non-monotone trace, so the corpus fails to build if either rejection ever silently stops happening.

Certification. `mfact certify release-manifest.json gates.json` exits 0 with `sorryFree`, `axiomsClean`, `fixturesPass`, and `evidenceComplete` all true. The genesis-folded release hash (BLAKE3, folded over all 397 artifact hashes in name order, seed `"mfact-v26.7.7-genesis"`) is `942facf3...f2d434` (truncated; full value in `release-manifest.json`). Both negative controls behaved as required: flipping `sorryFree` to false in a copy of the gates file causes `mfact certify` to exit 1 with an explicit gate-failure message; a syntactically corrupted manifest causes it to exit 2 with a typed refusal, not a crash.

Wall-clock. This section reports what the repository's git history actually timestamps: from the paper-scaffold commit to the certified-release commit, six commits span `2026-07-06T20:41:32-07:00` to `2026-07-06T21:09:48-07:00` (28 minutes), covering discovery and repair of three cross-family import/duplicate-declaration bugs, addition of the fixture pair, the axiom-audit reconciliation pass over 145 theorems, and certification. The ontology authoring and initial corpus rendering that preceded the first commit in this span are not independently timestamped in this repository's history; we report the receipted span only and do not extrapolate a total-run duration from it.

Final status. The table below is generated from the release manifest and standing records; no cell is hand-authored.

Release tag	`v26.7.7-procint-certified`
Release hash (BLAKE3 genesis fold)	`942facf32d48cd1a26c0f06b9396c6c150ab4d95d601bd090a8e1b9e7ef2d434`
Run identifier	`945bfca`
Lean build	PASS
Sorries / admits	0 / 0
Kernel-proven, axiom-audited theorems	197
Declarations (total / stated)	397 / 7
Axiom audit	PASS
Fixtures	PASS
Certified release	PASS
Standing Quadrature	PASS (kernel-admitted witness)
Crown-jewel WF-net theorem	PROVEN

13.2 Standing Quadrature

The release is not evaluated only by whether Lean builds. It is evaluated by whether the declaration catalog, admitted Lean corpus, process evidence, release manifest, and paper claims form a *closed standing graph*. The quadrature check rejects orphan declarations, unsupported claims, untraced artifacts, and manifest–paper divergence. Closure is itself kernel-checked: a generated witness module (`ProcInt.Release.Quadrature`) carries the name-sorted proven surfaces of the catalog, the manifest, and the axiom audit as literal lists, and proves their pairwise equality by `rfl` — any orphan on any side makes the lists differ and the kernel refuses the witness. The same module encodes the manufacturing run as a procint process trace and proves, with

the corpus's own `DeclareConstraint` semantics, that the run conforms to its declared process model: procint models the process by which mfact manufactured procint. Table 7 is generated from `quadrature.json`; three negative controls (a corrupted evaluation number, a claim with no evidence, a catalog declaration with no rendered symbol) each cause a typed refusal.

Surface pair	Count	Result
TTL to Lean	197	PASS
Lean to Audit	197	PASS
Audit to Manifest	197	PASS
Manifest to Paper	6	PASS
Artifact to Process Event	254	PASS
Claim to Evidence	5	PASS

Table 7: Standing Quadrature: PASS over 197 proven declarations, 5 traced paper claims, and 6 evaluation numbers. The closure is kernel-checked by the rendered witness `ProcInt.Release.Quadrature`.

13.3 praxis/rslab Benchmark Evidence

This section summarizes the empirical benchmark evidence collected for the `praxis-graphlaw` engine. The suite evaluates query performance, rule materialization latency, incremental delta processing, and cryptographic receipt validation. The headline metrics are presented in Table 8.

Benchmark Target	**Harness**	**Measured Value**
`test_transitive_rule`	bencher	875,808,862 ns
`graphlaw_materialize_delta` (mean)	divan	916.7 μs
`receipt_validate/1000` (point)	criterion	2.9059 ms

Table 8: Headline empirical metrics for the praxis-graphlaw release.

13.3.1 Micro-Benchmarks (Bencher)

13.3.2 Crate-Level & Control-Layer Benchmarks (Divan)

13.3.3 System-Level & Protocol Benchmarks (Criterion)

Profiling evidence was not collected because no profiler tooling exists in the praxis workspace as of this release.

Throughput measured; latency percentiles not yet collected; profiling not yet available.

14 Falsifier and Valid Objection Surface

A valid objection to this release must exhibit at least one of: (1) a declared admitted theorem contains `sorry`; (2) a theorem depends on unauthorized axioms; (3) a manifest hash does not match the artifact; (4) a negative fixture passes when it should fail; (5) a generated evaluation number does not match the manifest; (6) a stated declaration is described as proven; (7) the catalog-to-Lean rendering changes the intended theorem statement; (8) the release cannot be

Benchmark Name	Value
datalog_aggregate_facts_1000	1,557,120 ns
datalog_stratify_layers_20	13,308 ns
datalog_stratify_layers_200	132,318 ns
datalog_stratify_layers_50	33,184 ns
n3_chain_depth_150	29,828,662 ns
n3_chain_depth_400	471,124,433 ns
n3_chain_depth_50	1,873,179 ns
shacl_validate_100	327,036 ns
shacl_validate_1000	3,280,829 ns
shacl_validate_5000	16,661,195 ns
shacl_validate_complex_100	592,424 ns
shacl_validate_complex_1000	5,956,141 ns
shex_validate_100	104,714 ns
shex_validate_1000	1,111,820 ns
shex_validate_5000	5,654,160 ns
shex_validate_complex_100	104,392 ns
shex_validate_complex_1000	1,078,095 ns
shexc_parse_benchmark	3,416 ns
test_hierarchy_10	102,972 ns
test_hierarchy_100	1,006,702 ns
test_hierarchy_1000	10,316,249 ns
test_rdf_hierarchy_10	1,500,008 ns
test_transitive_rule	875,808,862 ns

Table 9: Bencher micro-benchmarks for praxis-graphlaw.

Benchmark Name	Fastest	Slowest	Median	Mean	Samples
action_precondition_mask	59.21 ns	107.30 ns	59.86 ns	60.47 ns	100
action_precondition_mask_condition_tree	97.30 ns	171.50 ns	99.89 ns	100.40 ns	100
bcinr_transition_table	0.62 ns	0.67 ns	0.63 ns	0.63 ns	100
graphlaw_materialize_delta	891.90 μs	953.70 μs	915.20 μs	916.70 μs	100
little_law_snapshot	46.84 ns	48.15 ns	47.49 ns	47.43 ns	100
pddl_action_filter	6.37 μs	15.58 μs	6.50 μs	6.79 μs	100
powl_step_tick	3.47 ns	3.80 ns	3.51 ns	3.51 ns	100
receipt_frame_link	243.10 ns	252.20 ns	247.00 ns	247.40 ns	100
standing_transition	20.15 ns	28.29 ns	20.47 ns	20.55 ns	100
verify_gate_dispatch	11.37 μs	13.87 μs	11.49 μs	11.54 μs	100

Table 10: Divan micro-benchmarks for praxis-graphlaw.

replayed under pinned dependencies. These are *closed failure classes*, not open attack surfaces: each is engineered into a gate, a typed refusal, or a non-admitted state, and the release gate is designed so that any occurrence of such a case prevents admission or produces a typed refusal. After certification, no constructor of the valid-objection surface is inhabited inside the declared boundary; the negative controls demonstrate each class refusing on a poisoned copy. A local objection to one proof, citation, or template is not automatically a refutation of mathematical manufacturing. It becomes a refutation only if it crosses the admission boundary and invalidates a claimed receipt — and the enumeration above is exactly the set of ways to do so.

Benchmark Name	Lower	Point Estimate	Upper
admission_throughput/admit_payload/green:	1.1792 µs	1.1809 µs	1.1829 µs
admission_throughput/judge_payload/green:	1.3706 µs	1.3720 µs	1.3733 µs
latency/emit_round_trip	186.4900 ns	187.1900 ns	187.9100 ns
latency/hash_bytes/small:	127.6000 ns	127.7700 ns	127.9600 ns
latency/serialize_payload:	50.9840 ns	51.0570 ns	51.1450 ns
receipt_validate/100	218.5300 µs	218.7800 µs	219.0200 µs
receipt_validate/1000	2.9027 ms	2.9059 ms	2.9095 ms
receipt_validate/500	1.2665 ms	1.2684 ms	1.2705 ms
scaling/emit_chain/chain_length/100:	29.0060 µs	29.0360 µs	29.0690 µs
scaling/emit_chain/chain_length/10:	2.8261 µs	2.8276 µs	2.8292 µs
scaling/emit_chain/chain_length/1:	215.3400 ns	215.7200 ns	216.1600 ns
scaling/emit_chain/chain_length/500:	146.5600 µs	146.8000 µs	147.0400 µs
throughput/hash_bytes/1024:	1.2284 µs	1.2289 µs	1.2295 µs
throughput/hash_bytes/16384:	10.2000 µs	10.2020 µs	10.2060 µs
throughput/hash_bytes/256:	348.9100 ns	349.9800 ns	351.4100 ns
throughput/hash_bytes/4096:	2.6420 µs	2.6429 µs	2.6440 µs
throughput/hash_bytes/64:	127.5100 ns	127.5900 ns	127.6900 ns

Table 11: Criterion benchmark results for the release.

15 Limitations and Standing

Proven versus stated. The library's standing is exactly the manifest's proven/stated split; prose in this paper adds nothing to it. *Stated* declarations are formal statements whose proof obligations are open — they compile, they are citable as definitions of the intended property, and they carry no more authority than that.

Crown-jewel scope. The short-circuit soundness characterization is reported per direction. If only one direction is admitted at release time, the claim is that direction and nothing more; the converse remains a stated obligation in the manifest. The release-time classification of the short-circuit soundness characterization is **PROVEN** (ground truth: `ProcInt.crownJewel_status`); this fragment is generated from the catalog and reports exactly that ground-truth classification, whichever value it holds — never a hand-asserted status.

What admission does not check. Kernel admission plus the axiom audit establishes that admitted terms prove the stated theorems under the pinned axioms. It does not establish that the catalog-curated statements faithfully render the informal canon; that correspondence rests on citation-annotated statement curation against the primary sources and is exactly the kind of residual risk the semantic audits of [9] target. Fidelity of statements is a review obligation, not a theorem.

Scope of the novelty claim. "First mechanization" claims are scoped to a search of the Lean (Mathlib/CSLib), Coq, and Isabelle (AFP) ecosystems at run time; we claim a negative search result, not a proof of absence.

Standing at submission. The standing of the system is intentionally not inferred from this paper. At release time, the crown theorem rail, countermodel rail, correspondence rail, product/DX

rail, and paper rail each carry independent manifest status. The paper is a publication surface, not a source of truth; if paper prose and manifest status ever diverge, the manifest and receipts govern, not the prose.

Exclusions. This paper does *not* claim any of the following.

Exclusion	Reason
LLMs prove mathematics autonomously	LLMs are untrusted producers
procint formalizes all of process intelligence	v1 boundary is manifest-scoped
Kernel admission proves informal intent	statement fidelity is a review obligation
Rice's theorem is violated	arbitrary semantic correctness is not decided
External opinion enters the standing calculus	only receipted artifacts are parsed
Criticism outside the boundary is refuted	the empty-objection-surface claim is scoped to the declared boundary
ggen output has standing by generation	only admitted output has standing
A stated theorem is proven	stated means the obligation remains open

16 Availability

The mfact and procint packages, the ontology, the templates, the release manifest, and the source of this paper (including the generated evaluation) will be made available in the `mfact` repository at the certified release tag accompanying this submission, pinned to Lean 4 v4.31.0 with Mathlib and CSLib at fixed revisions. The negative-control script that demonstrates the gate refusing a `sorry`-injected artifact ships with the release.

Reproducibility appendix.

- **Lean:** `leanprover/lean4:v4.31.0`.
- **Mathlib:** `leanprover-community/mathlib4` at the commit tagged `v4.31.0`.
- **CSLib:** `leanprover/cslib`, pinned to its last commit on the `v4.31.0` toolchain.
- **Build:** `lake build` in `mfact/` and `procint/`.
- **Rendering:** `ggen sync run` against `packs/lean-math-pack` and the merged `procint:` catalog. This renders the candidate Lean corpus (modules, index, axiom audit, registry tables) from the TTL catalog; the rendered corpus is admitted only by the subsequent `lake build`.
- **Certification:** `lake exe mfact certify release-manifest.json gates.json`.
- **Artifact and manifest hashes:** recorded per-declaration (BLAKE3) and as a genesis-folded release hash in `release-manifest.json`; reproduced in the generated evaluation (Section 13).
- **Expected output:** `CERTIFIED_RELEASE=PASS` with `SORRY_COUNT=0` and `AXIOM_AUDIT=PASS`; a negative control with an injected `sorry` must instead exit refused.

Replay. The release is tag `v26.7.7-procint-certified` (release hash `942facf32d48cd1a26c0f06b9396c6c150ab4d95d6`). A clean replay regenerates the corpus and re-certifies it:

Step	Command
Render corpus from catalog	`just render`
Build + audits (both packages)	`just build && just audit`
Regenerate manifest + gates	`just manifest`
Certify (exit 0 required)	`just certify`
Standing Quadrature	`just standing-quadrature`
Quadrature negative controls	`just quadrature-negative-controls`
arXiv package	`just arxiv-package`

The expected result is not "trust us"; it is a receipt: certification exits 0, the quadrature witness kernel-admits, and the negative controls refuse.

Publication packet. Publication of v26.7.7 is manufactured as its own identity, distinct from the certified core release: a packet hash folded over the packet's evidence inputs (recorded in `release/final_status.json`; this paper is itself one of those inputs and therefore cannot embed the hash without self-reference), with each actuation surface gated on evaluated requirements. The arXiv packet is `PENDING` (first pass (–plan): fragments not yet rendered; 13 declared files, declared = packed enforced). Packet statuses: `arxiv_upload`=`BLOCKED`, `github_push`=`BLOCKED`, `github_release`=`ALIVE`. No packet self-actuates: publication is recorded as `PENDING_EXTERNAL_ACTUATION` in every case, and the kernel-admitted witness `ProcInt.Release.PostRelease` refuses any render in which a packet claims otherwise.

Independent replay. The replay lane is upgraded only by its own gate's report, never asserted: its current standing is `REPLAY_PASS`. The gate clones the repository at tag `v26.7.7-procint-certified` into a scratch tree and re-runs the manufacturing rail there:

1. `git clone <stand-in>/mfact-dryrun.git at tag v26.7.7-procint-certified`
2. `just render && just build (pinned toolchain, lake exe cache get)`
3. `just audit && just test`
4. `just certify (exit 0 required)`
5. `just regen-check (ARTIFACT_DRIFT_REFUSED on any divergence)`

A replay that diverges on any ledgered artifact refuses with `ARTIFACT_DRIFT_REFUSED`; a replay that has not been run reports `REPLAY_NOT_RUN` rather than borrowing the core release's standing.

17 Conclusion

We have presented mfact as a Lean 4 framework for receipted mathematical manufacturing and procint as its first manufactured process-intelligence domain. The certified release `v26.7.7-procint-certified` contains 197 kernel-proven and axiom-audited theorems out of 397 declarations, with 0 sorries, positive and negative fixture coverage, and a BLAKE3-folded release hash. The central claim is not that generation is trustworthy; it is the opposite: generation is

untrusted, and therefore mathematical standing must be manufactured through explicit admission, refusal, audit, manifest, and replay.

This changes the unit of formalization. A theorem is no longer treated merely as a local proof term. In a manufactured release, a theorem is an admitted artifact with provenance, hash, axiom footprint, fixture status, manifest position, and replay path. A failed theorem is not erased or softened; it is refused or downgraded to stated status. A release is not declared successful by prose; it is certified by a receipt-bearing boundary. The release also closes Standing Quadrature (PASS): the declaration catalog, manifest, and audit surfaces, the claim–evidence ledger, and the process-trace conformance theorems are connected by generated witnesses admitted by Lean.

The crown-jewel workflow-net theorem is classified as *PROVEN*: the short-circuit characterization of soundness by liveness and boundedness (van der Aalst 1997, Lemma 8 / Theorem 11) is a kernel-admitted, axiom-audited theorem in this release, assembled from independently proven directions rather than asserted. This is not a release failure; it is the release boundary doing its job either way.

The broader result is a bounded response to the problem of generated formal content. Rice-style undecidability is not denied; arbitrary semantic judgment is replaced by declared admission inside a replayable boundary. Contact is not consequence; candidate generation is not proof; compilation is not full standing; and prose is not receipt. A mathematical artifact stands only when $R_B \vdash A = \mu(O_B^*)$. The release therefore does not ask for belief; it eliminates belief as a standing mechanism.

References

[1] Justin Asher. LeanExplore: A search engine for Lean 4 declarations, 2025.

[2] Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The lean computer science library, 2026.

[3] Alessandro Berti, Istvan Koren, Jan Niklas Adams, Gyunam Park, Benedikt Knopp, Nina Graves, Majid Rafiei, Lukas Liss, Leah Tacke genannt Unterberg, Yisong Zhang, Christopher Schwanen, Marco Pegoraro, and Wil M. P. van der Aalst. OCEL (Object-Centric Event Log) 2.0 specification, 2024.

[4] Josep Carmona, Boudewijn van Dongen, Andreas Solti, and Matthias Weidlich. *Conformance Checking: Relating Processes and Models*. Springer, 2018.

[5] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2024.

[6] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[7] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation, 2022.

[8] Humam Kourani and Sebastiaan J. van Zelst. POWL: Partially ordered workflow language. In *Business Process Management (BPM 2023)*, volume 14159 of *Lecture Notes in Computer Science*, pages 92–108. Springer, 2023.

[9] Theodore Meek, Siyuan Ge, Di Qiu Xiang, Simon Chess, and Vasily Ilin. Formalizing numerical analysis: An agent pipeline and quality audit beyond kernel acceptance, 2026.

[10] Micheline Bénédicte Moumoula, Serge Lionel Nikiema, Abdoul Kader Kabore, Jacques Klein, and Tegawendé F. Bissyande. Programming language confusion: When code LLMs can't keep their languages straight, 2025.

[11] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.

[12] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. APOLLO: Automated LLM and Lean collaboration for advanced formal reasoning, 2025.

[13] Jonathan Protzenko and Son Ho. Charon: A frontend for rust formal verification tools, 2024.

[14] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. Lean-auto: An interface between Lean 4 and automated theorem provers, 2025.

[15] Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean copilot: Large language models as copilots for theorem proving in Lean, 2024.

[16] The mathlib Community. The lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. arXiv:1910.09336.

[17] Wil M. P. van der Aalst. Verification of workflow nets. In *Application and Theory of Petri Nets 1997 (ICATPN 1997)*, volume 1248 of *Lecture Notes in Computer Science*, pages 407–426. Springer, 1997.

[18] Wil M. P. van der Aalst. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer, 2011.

[19] Wil M. P. van der Aalst and Alessandro Berti. Discovering object-centric Petri nets. *Fundamenta Informaticae*, 175(1–4):1–40, 2020. arXiv:2010.02047.

[20] Wil M. P. van der Aalst, Kees M. van Hee, Arthur H. M. ter Hofstede, Natalia Sidorova, H. M. W. Verbeek, Marc Voorhoeve, and Moe Thandar Wynn. Soundness of workflow nets: Classification, decidability, and analysis. *Formal Aspects of Computing*, 23(3):333–363, 2011.

[21] Ruida Wang, Jipeng Zhang, Yizhen Jia, Rui Pan, Shizhe Diao, Renjie Pi, and Tong Zhang. TheoremLlama: Transforming general-purpose LLMs into Lean4 experts, 2024.

[22] David Yin and Jing Gao. Generating millions of Lean theorems with proofs by exploring state transition graphs, 2025.